# Joint Server Selection and SFC Routing for Anycast in NFV-enabled SDNs

Huaqing Tu, Hongli Xu(✉), Liusheng Huang, Xuwei Yang, and Da Yao

University of Science and Technology of China, Hefei, China
{thq527,issacyxw,yddzf}@mail.ustc.edu.cn, {xuhongli,lshuang}@ustc.edu.cn

**Abstract.** Anycast mechanism has been used as a fundamental technology in many fields, with multiple servers providing the same service. Each request does not care which server it is served by. For security and performance reasons, requests must meet the requirements of service function chain (SFC) in network function virtualization (NFV) enabled networks. Though the routing of SFC has been widely studied, most of these works focus on unicast, which does not involve server selection, and cannot be applied directly for anycast. If we simply combine the traditional anycast algorithms and SFC unicast routing algorithms, it will overload the links and servers. Since overloaded links and servers will reduce the quality of services, it is important to balance the load of links and servers. In this work, we focus on designing an SFC routing mechanism for anycast that takes server selection into account and achieves load balancing among servers as well as links with the help of software defined network (SDN). We first propose the problem of joint server selection and SFC routing for anycast in NFV-enabled SDNs (JSR). Then based on the method of random rounding, we design an algorithm called RBLB with approximate ratio guarantee that can solve the JSR problem. The extensive simulation results show that our algorithm can reduce the load of servers and links. For example, algorithm RBLB can reduce the server load by 41.4%–60.1% compared with the comparison algorithm, when the number of requests exceeds 20k.

**Keywords:** Software defined network · Network function virtualization · Anycast routing

## 1 Introduction

The software defined network (SDN) [12] technology is a new networking paradigm which is designed to separate the control logic from the data plane. It can explore a higher utilization of links and servers by scheduling flows smartly through logically centralized control [9]. With these advantages of SDN, operators can manage the network flexibly [10] and compute forwarding paths of network flows to achieve certain objectives in data-centers and WAN [6,11].

The network function virtualization (NFV) is another kind of network technology, which uses virtualization technology to implement network functions

© Springer Nature Switzerland AG 2020
D. Yu et al. (Eds.): WASA 2020, LNCS 12384, pp. 432–443, 2020.
https://doi.org/10.1007/978-3-030-59016-1_36

such as firewall and WAN load-balancers, which play an important role in enhancing performance and ensuring security [14,17]. Typically, network flows need to traverse multiple virtual network functions (VNF) in a certain order, which is called SFC [15]. With the help of SDN and the trend of NFV, it is easy for operators to manage the network flexibly [10] and design routing path to pursue certain objectives in managing the traffic flows [6,11].

There are many studies pointing out the importance of anycast routing, such as resource/service requirement. In the definition of anycast routing, it allows multiple servers in the network to provide the same services. Each request can be served by one of these servers and does not particularly care which server it is served by. For example, Content Distribution Network (CDN) deploys multiple proxy servers across the network. Users request resources from the server, but they do not care which server the resources come from. For security and performance reasons, requests must traverse some network functions before reaching the server, which increases the complexity of anycast algorithm and may lead to poor performance. Thus, it is necessary to design an efficient anycast mechanism with SFC requirements.

However, since the traditional anycast routing algorithms, such as SSPF [20], does not consider SFC requirements when determining the routing path, they can not be applied directly for requests with SFC requirements. The state of art research of SFC routing, such as SIMPLE [14] and PDA [7], mainly focus on unicast. Since these algorithms do not involve server selection, they can not be applied directly in anycast. One may think that it is feasible that first determine the closest server for each request, and then use SFC unicast routing algorithm, which converts anycast to unicast. However, this heuristic method separates server selection from SFC routing in anycast, which can not provide provable performance guarantee. At the same time, this method makes the actual path length longer than expected, which will increase the load on the links. According to the experiment in Sect. 4, this method also causes overloaded links and servers. Therefore, it is important to design an efficient anycast mechanism that takes the SFC requirements into consideration.

In an anycast system, when a large number of requests for resources are generated, it is important to design an effective routing algorithm to avoid link and server congestion. Take an anycast routing algorithm called SSPF as an example, in which each request will be directed to the closest server using shortest path algorithm for resource access. Even if some links and servers are overloaded, this routing method may still direct requests to the overloaded links and servers. Previous work in [18] studies high throughput anycast routing in SDN network, which does not concern SFC requirements. Different from this work, our proposed solution concentrates on anycast routing with SFC requirements and achieves load balancing among servers as well as links.

We summarize our contributions as follows. In this paper, we first present the problem of designing of anycast system with SFC requirements in NFV-enabled SDNs. Different from the previous anycast mechanism, this anycast system meet SFC requirement and provide an efficient anycast routing mechanism, which can

achieve load balancing among servers as well as links. We propose the problem of joint server selection and SFC routing for anycast in NFV-enabled SDNs (JSR). We formulate this problem as an integer linear program, and design an algorithm called RBLB with approximation guarantee based on random rounding to solve JSR problem. We also verify the performance of our algorithm through simulation. Compared with the comparison algorithms, the algorithm can reduce the maximum load of servers by 41.4%–60.1%, when the number of requests exceeds 20k.

The rest of this paper mainly consists of the following sections. In Sect. 2, we analyze and define the problem of joint server selection and SFC routing for anycast in NFV-enabled SDNs. In Sect. 3, an algorithm called RBLB is proposed to deal with this problem. In Sect. 4, the simulation result are reported. Section 5 concludes the paper.

## 2 Preliminaries and Problem Definition

### 2.1 Network and Flow Models

An anycast system is composed of five different device sets: a host sets $H = \{h_1, ..., h_m\}$, with $m = |H|$; a server set $S = \{s_1, ..., s_n\}$, with $n = |S|$; an SDN switch set $V = \{v_1, ..., v_p\}$, with $p = |V|$; a VNF instance set $F = \{f_1, ..., f_q\}$, with $q = |F|$ and a cluster of controllers. Instances of all different network functions are included in VNF instance set $F$. In an SDN-enabled network, the controller only participates in selecting route paths for all flows and not be responsible for the packet forwarding. Thus, we model the NFV-enabled SDN network topology as $G = \{H \cup S \cup V \cup F, E\}$, where $E$ is the set of link which connect these devices. Note that, the servers cannot be directly connected, but connected through SDN switches.

Each host can generate some requests which need to be served by only one of servers in $S$. The set of requests in the network is denoted by $\Gamma$. For simplicity and ease of flow management, we assume that all requests are unsplitted. For each request $\gamma \in \Gamma$, the information including the SFC requirement, the traffic size and the duration can be collected by controllers. With the help of flow statistics collection supported by OpenFlow protocol, controllers can get all information mentioned before.

### 2.2 Problem Definition

We will provides more precise description of the problem, which is joint server selection and SFC routing for load balancing (JSR). According to the definition of anycast routing, a request generated by a host $h \in H$ can be served by only one server, say $s$. We denote $P_{\gamma,s}$ as the path set of for request $\gamma$ which will be served by server $s \in S$. Similar to the work in [7], each path $P_{\gamma,s}$ passes through VNF instances in a particular order required by its SFC requirement. The traffic intensity of request $\gamma$ is denoted as $f(\gamma)$. For the ease of expression, we regard the

CPU usage as the main resource constraint of servers. Other types of resource (e.g., memory, bandwidth) can be handled similarly [8]. Since the computation power of each server is limited, we denote the computation capacity of server $s \in S$ as $c(s)$. Let $c(e)$ be the maximum bandwidth load of link $e \in E$. With the help of logically centralized control plane, the SDN controllers can get the information of the load on all the links and servers. We denote the network load ratio as $\lambda = \{\frac{l(e)}{c(e)}, \frac{l(s)}{c(s)}, e \in E, s \in S\}$, where $l(e)$ and $l(s)$ are the load of link $e$ and server $s$, respectively. Note that, we do not include the load ratio of VNF instance in network load ratio, since the number and capacity of VNF instance can be changing with network load varying within the certain time [16]. We aim to reduce the maximum load of links and servers, that is, $\min \lambda$. The JSR problem is formulated as follows:

$$\min \quad \lambda$$

$$S.t. \begin{cases} \sum_{s \in S} x_\gamma^s = 1, & \forall \gamma \in \Gamma \\ \sum_{p \in P_{\gamma,s}} y_{\gamma,s}^p = x_\gamma^s, & \forall \gamma \in \Gamma, s \in S \\ \sum_{\gamma \in \Gamma} x_\gamma^s \cdot f(\gamma) \leq \lambda \cdot c(s), & \forall s \in S \\ \sum_{\gamma \in \Gamma} \sum_{s \in S} \sum_{e \in p: p \in P_{\gamma,s}} y_{\gamma s}^p \cdot f(\gamma) \leq \lambda \cdot c(e), & \forall e \in E \\ \sum_{\gamma \in \Gamma} \sum_{s \in S} \sum_{m \in p: p \in P_{\gamma,s}} y_{\gamma,s}^p \cdot f(\gamma) \leq c(m), & \forall m \in M \\ x_\gamma^s \in \{0,1\}, & \forall \gamma, s \\ y_{\gamma,s}^p \in \{0,1\}, & \forall \gamma, s, p \end{cases} \tag{1}$$

Where $x_\gamma^s$ denotes whether request $\gamma$ will be served by server $s$ or not, and $y_{\gamma,s}^p$ denotes whether the path $p \in P_{\gamma,s}$ will be selected for request $\gamma$ or not when $\gamma$ will be served by server $s$. The first set of equations represents that request $\gamma$ can only choose at most one server to provide service. The second set of equations means that the path $p \in P_{\gamma,s}$ can be chosen if and only if this request will be served by server $s$. The third and forth sets of inequations denote the link and server capacity constraints, respectively. The fifth set of inequations denotes that the load on each VNF instance can not violate its limitation.

**Theorem 1.** *The load-balancing anycast routing and service function chaining problem (defined in Eq. (1)) is NP-hard.*

The proof is omitted here due to space limit.

## 3 Algorithm Design

### 3.1 Rounding-Based Algorithm for JSR

To solve JSR problem, we design an algorithm called RBLB based on random rounding. We first construct a relaxation of JSR problem as a linear program described in Eq. (2). Specifically, the variables $\{x_\gamma^s\}$ and $\{y_{\gamma,s}^p\}$ are relaxed to be fractional, which means that a request can be served by several servers and

the traffic of each request can be arbitrarily splitted on more than one feasible path. The relaxed version linear program $LP_1$ is formulated as follows:

$$\min \quad \lambda$$

$$S.t. \begin{cases} \sum_{s\in S} x_\gamma^s = 1, & \forall \gamma \in \Gamma \\ \sum_{p\in P_{\gamma,s}} y_{\gamma,s}^p = x_\gamma^s, & \forall \gamma \in \Gamma, s \in S \\ \sum_{\gamma\in\Gamma} x_\gamma^s \cdot f(\gamma) \le \lambda \cdot c(s), & \forall s \in S \\ \sum_{\gamma\in\Gamma} \sum_{s\in S} \sum_{e\in p: p\in P_{\gamma,s}} y_{\gamma s}^p \cdot f(\gamma) \le \lambda \cdot c(e), & \forall e \in E \\ \sum_{\gamma\in\Gamma} \sum_{s\in S} \sum_{m\in p: p\in P_{\gamma,s}} y_{\gamma,s}^p \cdot f(\gamma) \le c(m), & \forall m \in M \\ x_\gamma^s \in [0,1], & \forall \gamma, s \\ y_{\gamma,s}^p \in [0,1], & \forall \gamma, s, p \end{cases} \tag{2}$$

The RBLB algorithm is comprised of two steps. We first solve the linear program described in Eq. (2) in the first step. Let $\{\widetilde{x}_\gamma^s\}$ and $\{\widetilde{y}_{\gamma,s}^p\}$ be the optimal solution. Since we have polynomial variables and constraints, this step can be finished in polynomial time. We assume that the optimal result is denoted as $\widetilde{\lambda}$.

The second step determines how to select a proper server and a efficient SFC routing for each request. By using the randomized rounding method, we can get the integer solutions $\{\hat{x}_\gamma^s\}$ and $\{\hat{y}_{\gamma,s}^p\}$ with $\gamma \in \Gamma, s \in S$ and $p \in P_{\gamma,s}$. For each request, the algorithm sets $\hat{x}_{\gamma^s}$ to 1 with probability $\widetilde{x}_\gamma^s$ independently. If $\hat{x}_\gamma^s = 1$, it suggests that request $\gamma$ will be processed by server $s$. Otherwise, request $\gamma$ will not be served by server $s$. After determining which server to serve the request $\gamma$, the algorithm selects an efficient SFC routing path. It sets $\hat{y}_{\gamma,s}^p$ to 1 with probability $\frac{\widetilde{y}_{\gamma,s}^p}{\widetilde{x}_\gamma^s}$, independently. If $\hat{y}_{\gamma,s}^p = 1$, it suggests that the path $p$ is selected for request $\gamma$. If $\hat{y}_{\gamma,s}^p = 0$, it means that the path $p$ will not be chosen. The RBLB algorithm is formally described in Algorithm 1.

**Algorithm 1.** RBLB: Rounding-Based Algorithm for JSR Problem

**Step 1: Solving the Relaxed JSR Problem**
Construct a linear program in Eq. (2)
Obtain the optimal solution $\{\widetilde{x}_\gamma^s\}$ and $\{\widetilde{y}_{\gamma,s}^p\}$
**Step 2: Determining Server and Anycast Routing**
Derive integer solutions $\{\hat{x}_\gamma^s\}$ and $\{\hat{y}_{\gamma,s}^p\}$ with $\gamma \in \Gamma$, $s \in S$ and $p \in P_{\gamma,s}$
**for** each request $\gamma \in \Gamma$ **do**
  Set $\hat{x}_\gamma^s$ to 1 with probability $\widetilde{x}_\gamma^s$
  **for** each server $s \in S$ **do**
    **if** $\hat{x}_\gamma^s = 1$ **then**
      Set $\widetilde{y}_{\gamma,s}^p$ to 1 with probability $\frac{\widetilde{y}_{\gamma,s}^p}{\widetilde{x}_\gamma^s}$
    **end if**
  **end for**
**end for**
Select server $s$ for request $\gamma$ if $\hat{x}_\gamma^s = 1$ and choose path p as its SFC routing if $\hat{y}_{\gamma,s}^p = 1$

### 3.2 Performance Analysis of RBLB Algorithm

We will present the performance analysis of the proposed RBLB algorithm in this section. Our proof process will use the following two famous theorems.

**Theorem 2 (Union Bound).** *Given a countable set of $n$ events: $A_1, A_2, ...,$ $A_n$, each event $A_i$ happens with possibility $\boldsymbol{Pr}(A_i)$. Then, $\boldsymbol{Pr}(A_1 \cup A_2 \cup ... \cup A_n) \leq \sum_{i=1}^{n} \boldsymbol{Pr}(A_i)$.*

**Theorem 3 (Chernoff Bound).** *Given $n$ independent variables: $y_1, y_2, ..., y_n$, where $\forall z_i \in [0, 1]$. Let $\mu = \mathbb{E}[\sum_{i=1}^{n} y_i]$. Then, $\boldsymbol{Pr}[\sum_{i=1}^{n} y_i \geq (1+\epsilon)\mu] \leq e^{\frac{-\epsilon^2 \mu}{2+\epsilon}}$, in which $\epsilon$ is an arbitrary value greater than 0.*

In the following, we first prove that, after randomizing the optimal solution, the algorithm still can guarantee that only one SFC routing path is selected for each request after determining which server will serve this request.

**Lemma 1.** *Only one SFC routing path is selected for each request after the randomized rounding process.*

*Proof.* On lines 4 to 14 of the algorithm, for each request $\gamma$, a server $s$ is chosen with probability $\widetilde{x}_\gamma^s$. After the server has been determined for this request, an SFC routing path $p \in P_{\gamma s}$ is selected by algorithm with probability $\frac{\widetilde{y}_{\gamma,s}^p}{\widetilde{x}_\gamma^s}$. If we can get that the total probability assigned for all feasible paths of each request is 1, it means that only one SFC routing path is selected.

According to the description of the proposed RBLB algorithm, the probability that SFC routing path $p$ is selected is:

$$\begin{aligned} \mathbf{Pr}[\hat{y}_{\gamma,s}^p = 1] &= \mathbf{Pr}[\hat{y}_{\gamma,s}^p = 1 | \hat{x}_\gamma^s = 1] \cdot \mathbf{Pr}[\hat{x}_\gamma^s = 1] \\ &\quad + \mathbf{Pr}[\hat{y}_{\gamma,s}^p = 1 | \hat{x}_\gamma^s = 0] \cdot \mathbf{Pr}[\hat{x}_{\gamma s} = 0] \\ &= \frac{\widetilde{y}_{\gamma,s}^p}{\widetilde{x}_\gamma^s} \cdot \widetilde{x}_\gamma^s = \widetilde{y}_{\gamma,s}^p \end{aligned} \tag{3}$$

Then, the total probability on all paths is:

$$\begin{aligned} \sum_{s \in S} \sum_{p \in P_{\gamma,s}} \mathbf{Pr}[\hat{y}_{\gamma,s}^p = 1] &= \sum_{s \in S} \sum_{p \in P_{\gamma s}} \widetilde{y}_{\gamma,s}^p \\ &= \sum_{s \in S} \widetilde{x}_\gamma^s = 1 \end{aligned} \tag{4}$$

Note that, the first, second and third equations hold according to Eq. (3) and equations described in Eq. (2). Equation (4) shows that a SFC routing path will be selected for each request after the randomized rounding process.

For the convenience of the following proof, we denote the minimum capacity of all servers, links and VNF instances as $c^{min}(s)$, $c^{min}(e)$ and $c^{min}(f)$, respectively. The whole request set is denoted as $\Gamma$. Variable $\alpha$ is defined as follows:

$$\alpha = \min\{\frac{c^{min}(s)\cdot\widetilde{\lambda}}{f(\gamma)}, \frac{c^{min}(e)\cdot\widetilde{\lambda}}{f(\gamma)}, \frac{c^{min}(m)}{f(\gamma)}, \gamma \in \Gamma\} \tag{5}$$

It can be observed that $\alpha \gg 1$ is true under many practical situations [2,19]. Even if the RBLB algorithm is a randomized rounding algorithm, we can still calculate the expected load on each server, link and VNF instance. Next, we give the performance analysis on server capacity constraints.

**Theorem 4.** *The proposed RBLB algorithm ensures that the total load on any server $s \in S$ is less than the load of the fractional solution by a factor of $\frac{3\log n}{\alpha}+3$*

*Proof.* We express the load generated by request $\gamma$ on server $s$ as $\varphi_{\gamma,s}$, then the expectation of variable $\varphi_{\gamma,s}$ is:

$$\mathbb{E}[\varphi_{\gamma,s}] = \mathbf{Pr}[\hat{x}^s_\gamma = 1]\cdot f(\gamma) = \widetilde{x}^s_\gamma \cdot f(\gamma) \tag{6}$$

Then, combining Eq. (6) and the third set of inequations in Eq. (2), we can get the total expected load on server $s$ is:

$$\mathbb{E}\left[\sum_{\gamma\in\Gamma}\varphi_{\gamma,s}\right] = \sum_{\gamma\in\Gamma}\mathbb{E}[\varphi_{\gamma,s}] = \sum_{\gamma\in\Gamma}\widetilde{x}^s_\gamma\cdot f(\gamma) \le \widetilde{\lambda}\cdot c(s) \tag{7}$$

According to the definition of $\alpha$ in Eq. (5), we have:

$$\begin{cases} \dfrac{\varphi_{\gamma,s}\cdot\alpha}{\widetilde{\lambda}\cdot c(s)} \in [0,1] \\ \mathbb{E}\left[\sum_{\gamma\in\Gamma}\dfrac{\varphi_{\gamma,s}\cdot\alpha}{\widetilde{\lambda}\cdot c(s)}\right] \le \alpha \end{cases} \tag{8}$$

Then applying Theorem 3, we get the following formula:

$$\mathbf{Pr}\left[\sum_{\gamma\in\Gamma}\frac{\varphi_{\gamma,s}\cdot\alpha}{\widetilde{\lambda}\cdot c(s)} \ge (1+\xi)\alpha\right] \le e^{\frac{-\xi^2\alpha}{2+\xi}} \tag{9}$$

where $\xi$ is an arbitrary positive value. Now, we assume that:

$$\mathbf{Pr}\left[\sum_{\gamma\in\Gamma}\frac{\alpha\cdot\varphi_{\gamma,s}}{\widetilde{\lambda}\cdot c(s)} \ge (1+\xi)\alpha\right] \le e^{\frac{-\xi^2\alpha}{2+\xi}} \le \frac{\mathcal{P}}{n} \tag{10}$$

where $\mathcal{P}$ is a variable related to the size of the network. We can use the number of switches to indicate the size of the network. When the network scale increase, $\mathcal{P} \to 0$.

By solving the inequality in Eq. (10) and setting $\mathcal{P} = \frac{1}{n^2}$, we have:

$$\begin{aligned} \xi &\ge \frac{\log\frac{n}{\mathcal{P}} + \sqrt{\log^2\frac{n}{\mathcal{P}} + 8\alpha\log\frac{n}{\mathcal{P}}}}{2\alpha} \\ &\Rightarrow \xi \ge \frac{3\log n}{\alpha} + 2, \quad n \ge 2 \end{aligned} \tag{11}$$

Then, applying Theorem 2 and following Eq. (10), we can get that:

$$\begin{aligned}
&\mathbf{Pr}\left[\bigvee_{s\in S}\sum_{\gamma\in\Gamma}\frac{\varphi_{\gamma,s}}{\widetilde{\lambda}\cdot c(s)}\geq(1+\xi)\right]\\
&\leq\sum_{s\in S}\mathbf{Pr}\left[\sum_{\gamma\in\Gamma}\frac{\varphi_{\gamma,s}}{\widetilde{\lambda}\cdot c(s)}\geq(1+\xi)\right]\\
&\leq|S|\cdot\frac{1}{n^3}\leq\frac{1}{n^2},\quad \xi\geq\frac{3\log n}{\alpha}+2
\end{aligned}\tag{12}$$

Then, following Eq. (12), we can get $1+\xi=\frac{3\log n}{\alpha}+3$, which means that the total load on server $s$ is no more than the load of the fractional solution by a factor of $\frac{3\log n}{\alpha}+3$ after the randomized rounding process.

**Theorem 5.** *The RBLB algorithm ensures that the total traffic load on any link $e\in E$ is no more than the traffic of the fractional solution by a factor of $\frac{4\log n}{\alpha}+3$.*

Since the space is limited and the proof of Theorem 5 and Theorem 4 are similar, the proof of Theorem 5 is omitted here.

**Theorem 6.** *The RBLB algorithm ensures that the total load on any VNF instance $f\in F$ is no more than the capacity of this VNF instance a factor of $\frac{3\log n}{\alpha}+3$.*

Similar to Theorem 5, the proof of Theorem 6 is omitted.

## 4 Performance Evaluation

We compare the RBLB algorithm with the comparison algorithms to check the performance and correctness of the RBLB algorithm in this section. We first introduce the simulation settings (Sect. 4.1), then give the benchmarks (Sect. 4.2). After that, through extensive simulations, we evaluate the load balancing anycast routing algorithm with several comparison algorithms (Sect. 4.3).

### 4.1 Simulation Setting

To objectively evaluate the performance of RBLB algorithm, we choose two typical and practical topologies to verify the performance of the algorithm. The first topology is the fat-tree topology [1], denoted as (a), which contains 80 switches, 118 hosts and 10 servers. The fat-tree topology has been applied in many data center networks. The second topology is Monash campus network, denoted as (b), which contains 100 switches, 180 hosts and 10 servers. The capacity of each server is set as 1 Gbps. The request set is generated randomly. In detail, the source host of a request is randomly picked in the network, and is SFC requirement is also randomly picked in network function set. For the

traffic size of request (or flow), the 20% of the top-ranked requests may produce more than 80% of the total traffic [3]. We allocate the size of each flow and each request according to this rule. The network function set contains 5 different kinks of VNFs, which is Firewall, IDS, Load-Balancer, Monitor, NAT. According to [13], the CPU cycles per packet of each VNF is 1348, 1348, 1348, 1676, 1631, respectively. We generate at most 32K requests in the network, and construct a layered graph to compute feasible path set for each request by using Dijkstras shortest path algorithm.

### 4.2 Benchmarks and Performance Metrics

In the simulations, different metrics are adopted to evaluate performance. The first one is the maximum server load, which is used to indicate the load balancing of servers. The second metric is maximum link load, which is used to indicate the load balancing of links. The last metric is maximum VNF load, which is used to reflect that the proposed algorithm can not only realize the load balancing of servers and links, but also reduce the maximum load of VNF instances. We compare it with two benchmarks to evaluate how well proposed algorithm performs. Since there are no previous solutions for the anycast routing with SFC requirement, the first benchmark, called nearest server routing (NSR), is modified on the basis of SSPF algorithm [20]. Specifically, for these requests, we first use SSPF algorithm to get the neatest server, then construct a layered graph and use Dijkstras shortest path algorithm [4,5] to get SFC paths. The second benchmark is called shortest SFC anycast routing (SAR). In detail, for each request, it uses algorithm in [5] to get the shortest path set of each server that meets SFC requirements, and then selects the shortest one as anycast routing.

### 4.3 Simulation Results

We observe the maximum server load by changing the number of requests from 8K to 32K in the network in the first set of simulations. Figure 1 and Fig. 2 show that the maximum traffic load on all servers increases with the growing number of requests in the network for all three algorithms on both topologies. The reason for this is that as the number of requests increases, so does the requests that need to be processed by the server. Thus, the resources consumed by each server will also increase. Specifically, in topology (a), when the number of requests is 20K, the maximum server load of RBLB, NSR and SAR is 251 Mbps, 664 Mbps and 708 Mbps, respectively. That is, RBLB can reduces the maximum server load ratio by 41.4% and 45.7% compared with NSR and SAR, respectively.

The second set of simulations applies three algorithms on two topologies. We observe changes in the maximum link load by altering the number of requests from 8K to 32K in the network. Figure 3 and Fig. 4 show the maximum link load on all links grows with the changing size of requests in the network for all three algorithms on both topologies. The reason for this is that when the number of requests grows, the total traffic load in the network also increases, and the load on each link also increases. Specifically, in topology (b), when the

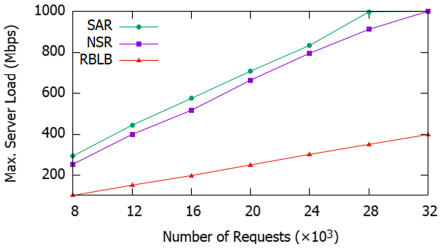


**Fig. 1.** Max. server load vs. number of requests for topology (a).

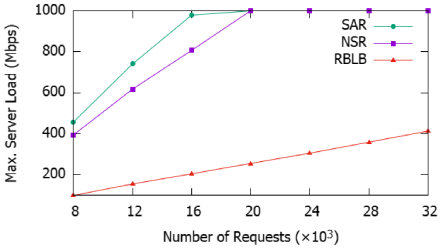


**Fig. 2.** Max. server load vs. number of requests for topology (b).

number of requests is 12K, the maximum link load of RBLB, NSR, SAR is 156 Mbps, 618 Mbps and 740 Mbps, respectively. That is, RBLB can cuts down the maximum load ratio of link by 46.2% and 58.4% compared with NSR and SAR, respectively.

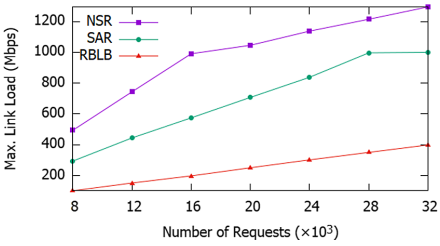


**Fig. 3.** Max. link load vs. number of requests for topology (a).

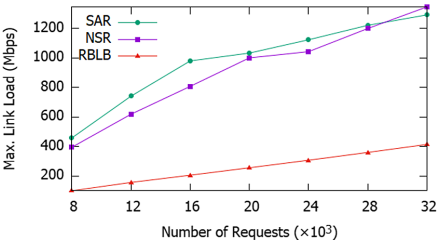


**Fig. 4.** Max. link load vs. number of requests for topology (b).

In third set of simulations, we observe the maximum load ratio of VNF instances by changing the number of requests from 8K to 32K in the network on two topologies. Figure 5 and Fig. 6 show the maximum load ratio on all VNF instances grows with the changing size of requests for all three algorithms on both topologies. The reason for this is that when the number of requests grows, each VNF instance needs to handle more requests, thus the maximum load ratio on VNF instances grows. Specifically, in topology (a), when the number of requests is 20K, the maximum load ratio of VNF instances for RBLB, NAR and SAR is 0.54, 1.02 and 0.82, respectively. That is, RBLB can reduces the maximum load ratio of all VNF instances by 48% and 32%, respectively.

Through the above three sets of simulation results, we can draw the following conclusions. First, RBLB algorithm achieve smaller maximum server load. For example, RBLB can reduces the maximum server load by 41.4% and 45.7% compared with NSR and SAR when there are 20K requests in topology (a). Second, RBLB can achieve smaller maximum link load. For example, given 12K requests in topology (b), the maximum link load is reduced by 46.2% and 58.4% compared with NSR and SAR. Third, in addition to reducing the maximum server

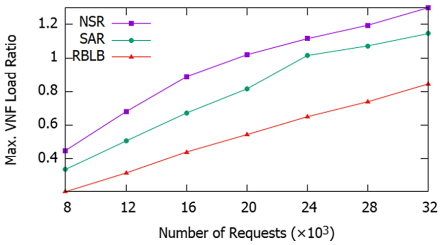


**Fig. 5.** Max. VNF load ratio vs. number of requests for topology (a).

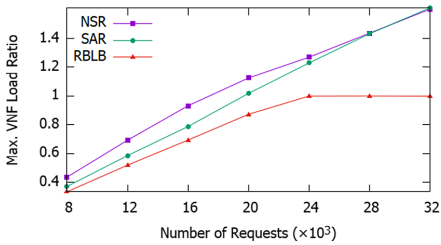


**Fig. 6.** Max. VNF load ratio vs. number of requests for topology (b).

and link load, RBLB also reduces the maximum load ratio of VNF instances. For example, RBLB can reduces the maximum load ratio of all VNF instances by 48% and 32% compared with NSR and SAR when there are 20K requests in topology (a). These data reflect the advantages of the proposed algorithm.

## 5 Conclusion

In this work, we analysis how to implement an anycast routing mechanism which satisfies SFC requirements and achieves link and server load balancing in NFV-enabled SDNs. This problem is formulated as an integer linear program. To solve this problem, we propose an algorithm with approximation guarantee based on random rounding. Through simulation results, the performance of proposed algorithm RBLB is better than the comparison algorithms.

## References

1. Al-Fares, M., Loukissas, A., Vahdat, A.: A scalable, commodity data center network architecture. ACM SIGCOMM Comput. Commun. Rev. **38**(4), 63–74 (2008)
2. Cohen, R., Lewin-Eytan, L., Naor, J.S., Raz, D.: On the effect of forwarding table size on SDN network utilization. In: IEEE Conference on Computer Communications, IEEE INFOCOM 2014, pp. 1734–1742. IEEE (2014)
3. Curtis, A.R., Mogul, J.C., Tourrilhes, J., Yalagandula, P., Sharma, P., Banerjee, S.: Devoflow: scaling flow management for high-performance networks. In: Proceedings of the ACM SIGCOMM 2011 Conference, pp. 254–265 (2011)
4. Dijkstra, E.W., et al.: A note on two problems in connexion with graphs. Numer. Math. **1**(1), 269–271 (1959)
5. Dwaraki, A., Wolf, T.: Adaptive service-chain routing for virtual network functions in software-defined networks. In: Proceedings of the 2016 Workshop on Hot Topics in Middleboxes and Network Function Virtualization, pp. 32–37 (2016)
6. Gember-Jacobson, A., et al.: OpenNF: enabling innovation in network function control. ACM SIGCOMM Comput. Commun. Rev. **44**(4), 163–174 (2014)
7. Guo, L., Pang, J., Walid, A.: Dynamic service function chaining in SDN-enabled networks with middleboxes. In: 2016 IEEE 24th International Conference on Network Protocols (ICNP), pp. 1–10. IEEE (2016)

8. Guo, L., Pang, J., Walid, A.: Joint placement and routing of network function chains in data centers. In: IEEE Conference on Computer Communications, IEEE INFOCOM 2018, pp. 612–620. IEEE (2018)
9. Hong, C.Y., et al.: Achieving high utilization with software-driven WAN. In: Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM, pp. 15–26 (2013)
10. Li, B., et al.: ClickNP: highly flexible and high performance network processing with reconfigurable hardware. In: Proceedings of the 2016 ACM SIGCOMM Conference, pp. 1–14 (2016)
11. Li, Y., Phan, L.T.X., Loo, B.T.: Network functions virtualization with soft real-time guarantees. In: The 35th Annual IEEE International Conference on Computer Communications, IEEE INFOCOM 2016, pp. 1–9. IEEE (2016)
12. Malboubi, M., Wang, L., Chuah, C.N., Sharma, P.: Intelligent SDN based traffic (de)aggregation and measurement paradigm (ISTAMP). In: IEEE Conference on Computer Communications, IEEE INFOCOM 2014, pp. 934–942. IEEE (2014)
13. Martins, J., et al.: ClickOS and the art of network function virtualization. In: 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2014), pp. 459–473 (2014)
14. Qazi, Z.A., Tu, C.C., Chiang, L., Miao, R., Sekar, V., Yu, M.: SIMPLE-fying middlebox policy enforcement using SDN. In: Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM, pp. 27–38 (2013)
15. Quinn, P., Nadeau, T.: Problem statement for service function chaining. In: RFC 7498. RFC Editor (2015)
16. Rajagopalan, S., Williams, D., Jamjoom, H., Warfield, A.: Split/merge: system support for elastic execution in virtual middleboxes. In: Presented as Part of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2013), pp. 227–240 (2013)
17. Sherry, J., Hasan, S., Scott, C., Krishnamurthy, A., Ratnasamy, S., Sekar, V.: Making middleboxes someone else's problem: network processing as a cloud service. ACM SIGCOMM Comput. Commun. Rev. **42**(4), 13–24 (2012)
18. Xu, H., Li, X., Huang, L., Wang, J., Leng, B.: High-throughput anycast routing and congestion-free reconfiguration for SDNs. In: 2016 IEEE/ACM 24th International Symposium on Quality of Service (IWQoS), pp. 1–6. IEEE (2016)
19. Xu, H., Yu, Z., Li, X.Y., Qian, C., Huang, L., Jung, T.: Real-time update with joint optimization of route selection and update scheduling for SDNs. In: 2016 IEEE 24th International Conference on Network Protocols (ICNP), pp. 1–10. IEEE (2016)
20. Xuan, D., Jia, W., Zhao, W.: Routing algorithms for anycast messages. In: Proceedings of 1998 International Conference on Parallel Processing (Cat. No. 98EX205), pp. 122–130. IEEE (1998)